K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Batch: H5	Roll No.: 16010122158
Experiment No. 6	

TITLE : To perform time series analysis on health care

AIM: To perform forecasting using time series analysis

Expected OUTCOME of Experiment:

CO4: Perform Time series Analytics and forecasting

Books/ Journals/ Websites referred:

Pre Lab/ Prior Concepts:

Students should have a basic understanding of: Time series Analytics and forecasting

Procedure:

Data set Used: Hospital_patients_datasets

Step1: Select and Load the dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from prophet import Prophet
```

```
# Load the dataset
file_path = 'Hospital_patients_datasets.csv'  # replace with actual path
data = pd.read_csv(file_path)
```

Step2: Convert 'ScheduledDay' and 'AppointmentDay' to datetime format

```
# Load the dataset
file_path = 'Hospital_patients_datasets.csv'  # replace with actual path
data = pd.read_csv(file_path)

# Convert 'ScheduledDay' and 'AppointmentDay' to datetime format
data['ScheduledDay'] = pd.to_datetime(data['ScheduledDay'])
data['AppointmentDay'] = pd.to_datetime(data['AppointmentDay'])

# Create new features: Appointment Weekday, Scheduled Weekday, Days Between Scheduling and Appointment
data['AppointmentWeekday'] = data['AppointmentDay'].dt.day_name()
data['ScheduledWeekday'] = data['ScheduledDay'].dt.day_name()
data['DaysBetween'] = (data['AppointmentDay'] - data['ScheduledDay']).dt.days

print(data)
```

\

```
           PatientId  AppointmentID Gender              ScheduledDay  \
0       2.987250e+13        5642903      F 2016-04-29 18:38:08+00:00
1       5.589980e+14        5642503      M 2016-04-29 16:08:27+00:00
2       4.262960e+12        5642549      F 2016-04-29 16:19:04+00:00
3       8.679510e+11        5642828      F 2016-04-29 17:29:31+00:00
4       8.841190e+12        5642494      F 2016-04-29 16:07:23+00:00
...              ...            ...    ...                       ...
110522  2.572130e+12        5651768      F 2016-05-03 09:15:35+00:00
110523  3.596270e+12        5650093      F 2016-05-03 07:27:33+00:00
110524  1.557660e+13        5630692      F 2016-04-27 16:03:52+00:00
110525  9.213490e+13        5630323      F 2016-04-27 15:09:23+00:00
110526  3.775120e+14        5629448      F 2016-04-27 13:30:56+00:00

                AppointmentDay  Age      Neighbourhood  Scholarship  \
0      2016-04-29 00:00:00+00:00   62    JARDIM DA PENHA            0
1      2016-04-29 00:00:00+00:00   56    JARDIM DA PENHA            0
2      2016-04-29 00:00:00+00:00   62      MATA DA PRAIA            0
3      2016-04-29 00:00:00+00:00    8  PONTAL DE CAMBURI            0
4      2016-04-29 00:00:00+00:00   56    JARDIM DA PENHA            0
...                          ...  ...                ...          ...
110522 2016-06-07 00:00:00+00:00   56        MARIA ORTIZ            0
110523 2016-06-07 00:00:00+00:00   51        MARIA ORTIZ            0
110524 2016-06-07 00:00:00+00:00   21        MARIA ORTIZ            0
110525 2016-06-07 00:00:00+00:00   38        MARIA ORTIZ            0
110526 2016-06-07 00:00:00+00:00   54        MARIA ORTIZ            0

        Hipertension  Diabetes  Alcoholism  Handcap  SMS_received No-show
```

```
        Hipertension  Diabetes  Alcoholism  Handcap  SMS_received No-show  \
0                  1         0           0        0             0      No
1                  0         0           0        0             0      No
2                  0         0           0        0             0      No
3                  0         0           0        0             0      No
4                  1         1           0        0             0      No
...              ...       ...         ...      ...           ...     ...
110522             0         0           0        0             1      No
110523             0         0           0        0             1      No
110524             0         0           0        0             1      No
110525             0         0           0        0             1      No
110526             0         0           0        0             1      No

       AppointmentWeekday ScheduledWeekday  DaysBetween
0                  Friday           Friday           -1
1                  Friday           Friday           -1
2                  Friday           Friday           -1
3                  Friday           Friday           -1
4                  Friday           Friday           -1
...                   ...              ...          ...
110522            Tuesday          Tuesday           34
110523            Tuesday          Tuesday           34
110524            Tuesday        Wednesday           40
110525            Tuesday        Wednesday           40
110526            Tuesday        Wednesday           40

[110527 rows x 17 columns]
```

Step 3: Initialize Prophet model for forecasting

```
# Step 3: Initialize Prophet model for forecasting
model = Prophet()
```

Step 4: Fit the model

```
# Step 4: Fit the model
model.fit(attendance_daily)
```

Step 5: Create future dates for prediction

```
# Step 5: Create future dates for prediction (e.g., next 30 days)
future_dates = model.make_future_dataframe(periods=30)
```

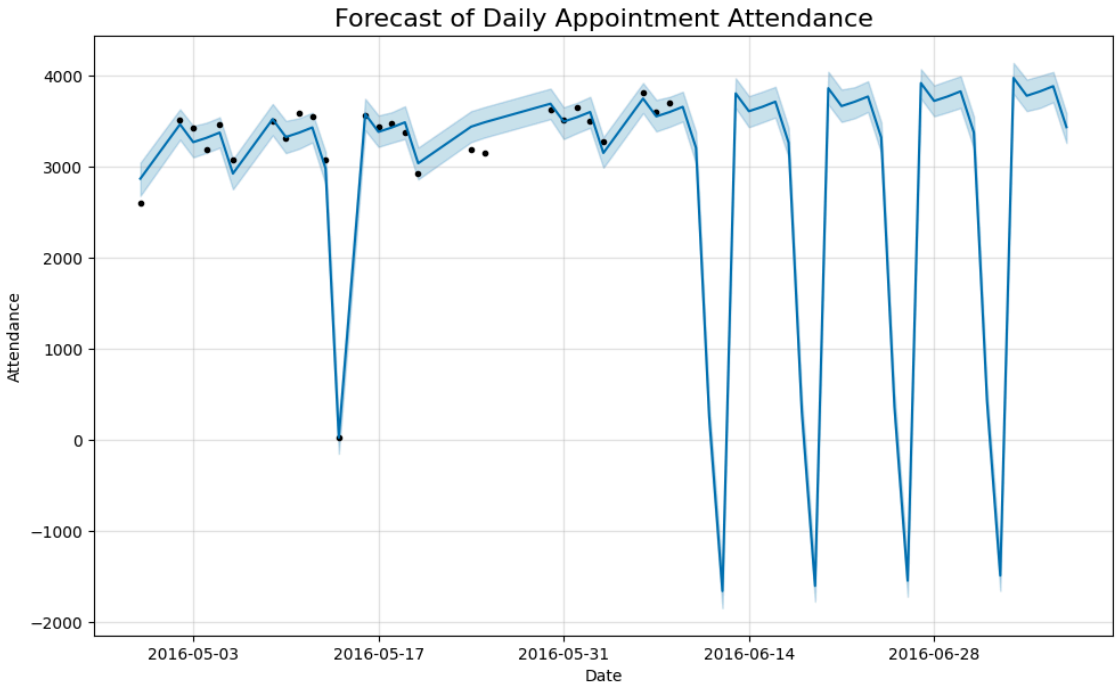
```
INFO:prophet:Disabling yearly seasonality. Run prophet with yearly_seasonality=True to override this.
INFO:prophet:Disabling daily seasonality. Run prophet with daily_seasonality=True to override this.
INFO:prophet:n_changepoints greater than number of observations. Using 20.
DEBUG:cmdstanpy:input tempfile: /tmp/tmp10mk5s1x/gx0afseb.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmp10mk5s1x/b8jheabk.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.10/dist-packages/prophet/stan_model/prophet_model.bin', 'random', 'seed=93340', 'data', 'file=/tmp/tmp10mk
11:40:10 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
11:40:10 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
```

Step 6: Predict future attendance

```
# Step 6: Predict future attendance
forecast = model.predict(future_dates)
```

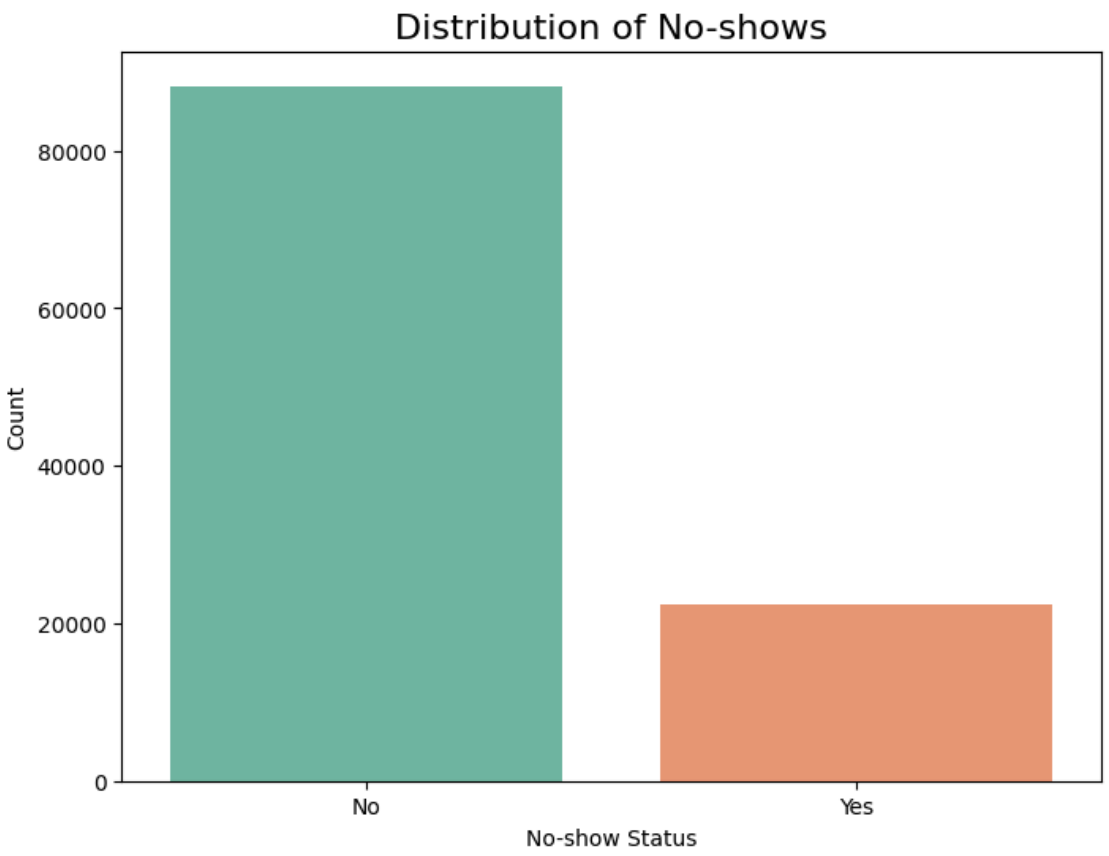
Step 7: Plot the forecast

```
# Step 7: Plot the forecast
fig = model.plot(forecast)
plt.title('Forecast of Daily Appointment Attendance', fontsize=16)
plt.xlabel('Date')
plt.ylabel('Attendance')
plt.show()
```

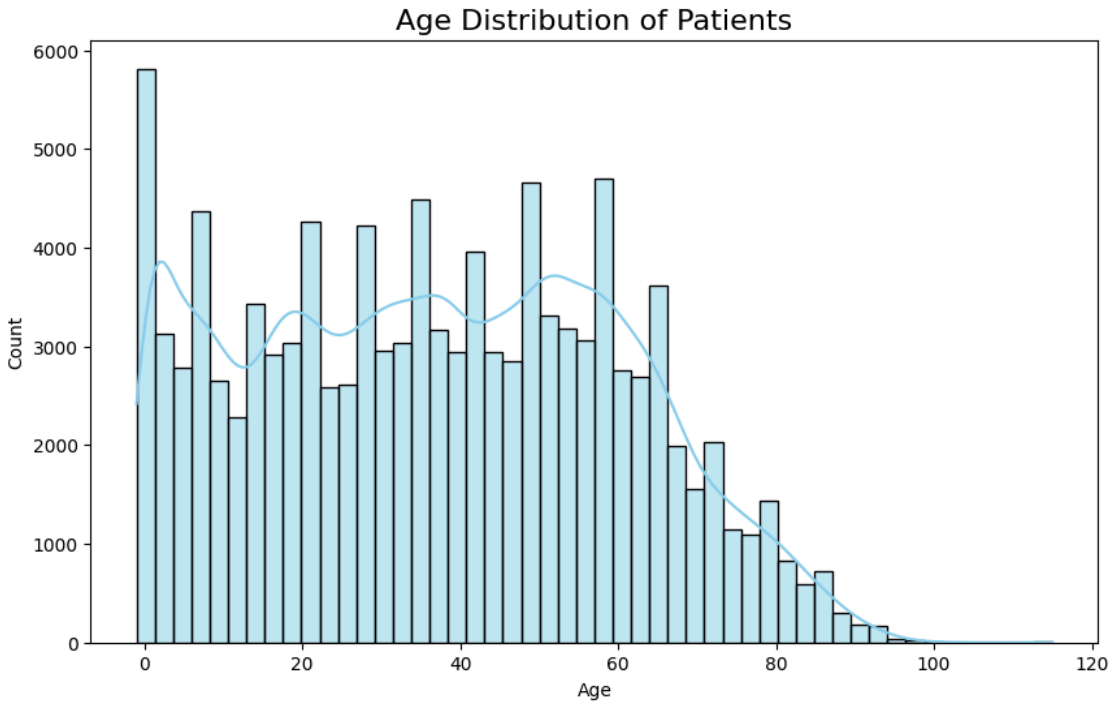


Step 8: Exploratory Data Analysis Functions and Running the analysis functions

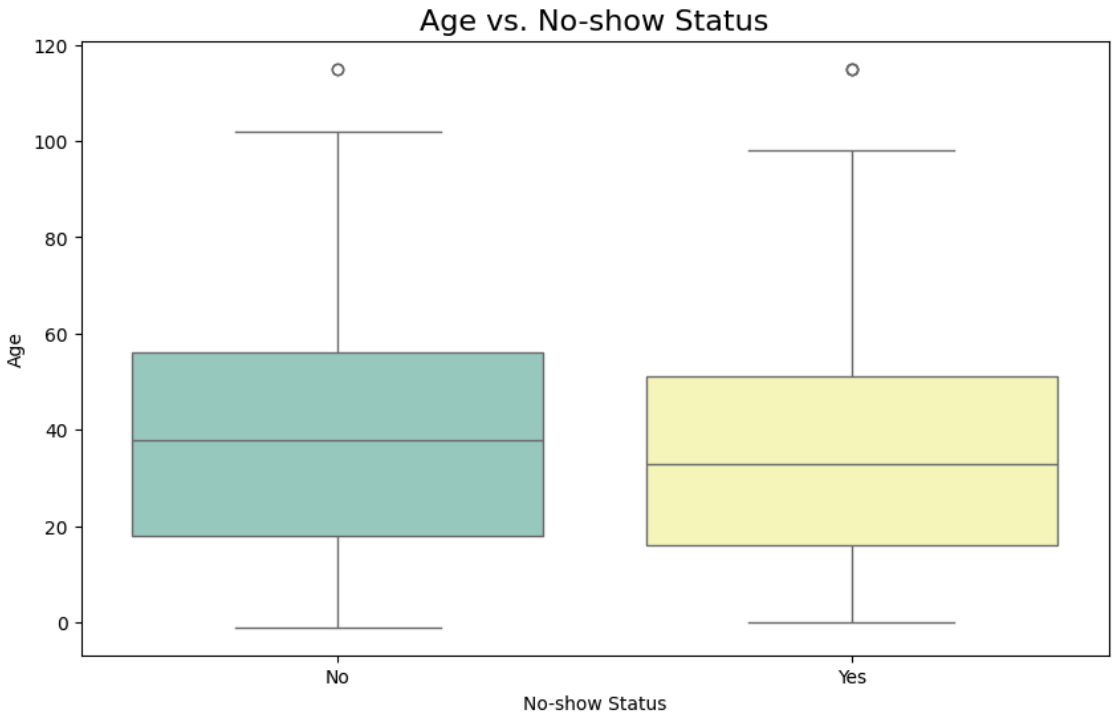
```
# Exploratory Data Analysis Function
def plot_no_show_distribution():
    """Plot no-show distribution"""
    plt.figure(figsize=(8, 6))
    sns.countplot(x='No-show', data=data, palette="Set2")
    plt.title('Distribution of No-shows', fontsize=16)
    plt.ylabel('Count')
    plt.xlabel('No-show Status')
    plt.show()


# Running the analysis function
plot_no_show_distribution()
```



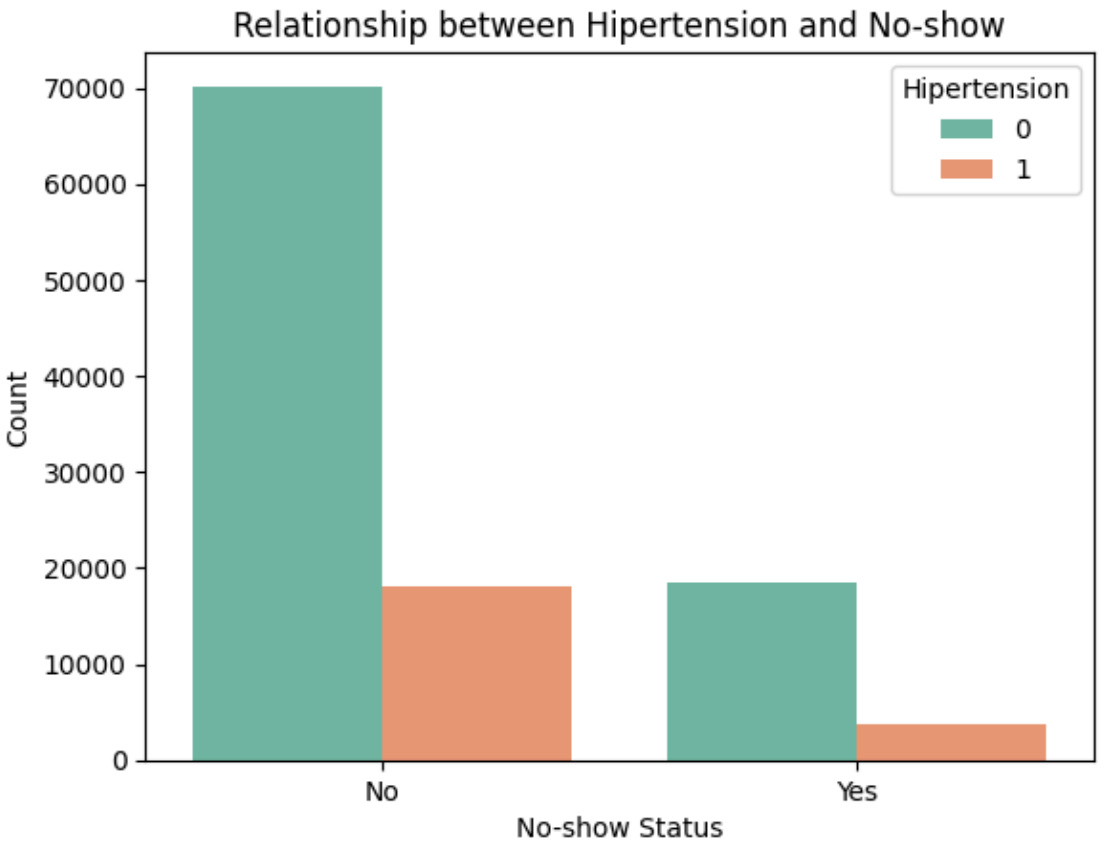
```python
# Exploratory Data Analysis Function
def plot_age_distribution():
    """Plot age distribution of patients"""
    plt.figure(figsize=(10, 6))
    sns.histplot(data=data, x='Age', bins=50, kde=True, color='skyblue')
    plt.title('Age Distribution of Patients', fontsize=16)
    plt.xlabel('Age')
    plt.ylabel('Count')
    plt.show()


# Running the analysis function
plot_age_distribution()
```

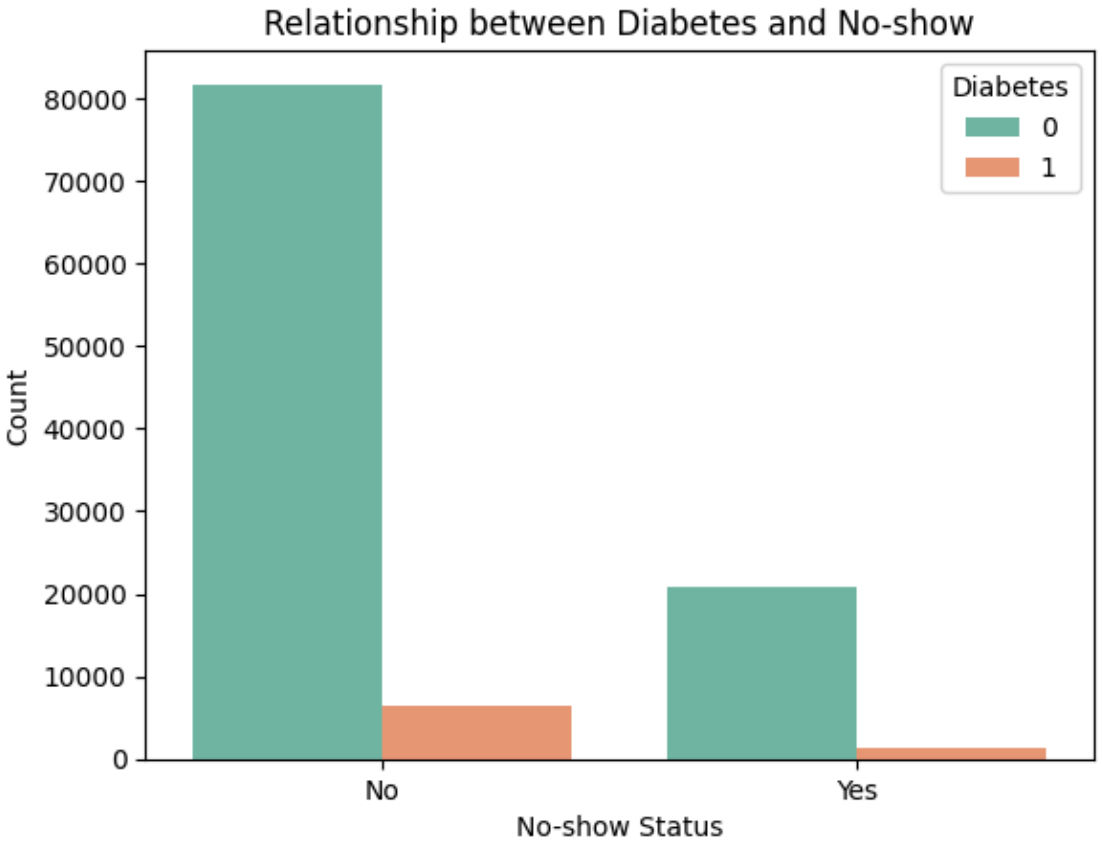


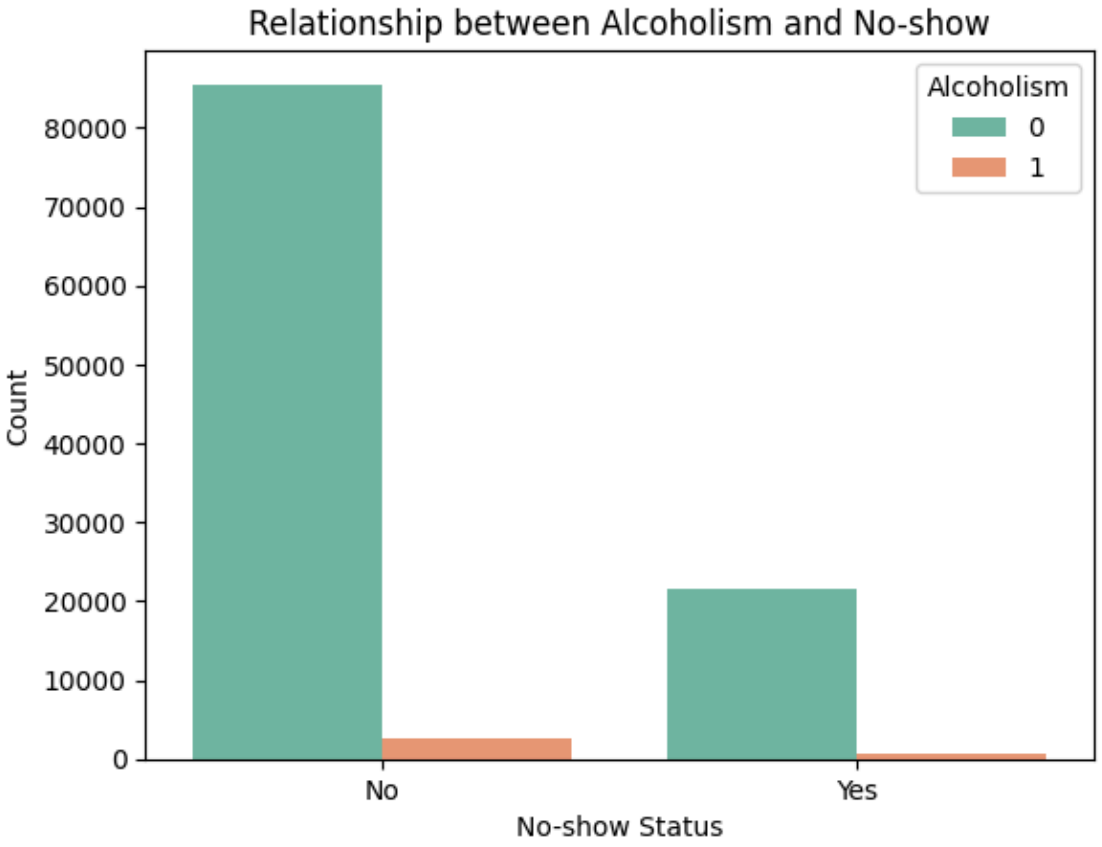
```python
# Exploratory Data Analysis Function
def plot_age_vs_no_show():
    """Plot relationship between Age and No-show status"""
    plt.figure(figsize=(10, 6))
    sns.boxplot(x='No-show', y='Age', data=data, palette="Set3")
    plt.title('Age vs. No-show Status', fontsize=16)
    plt.xlabel('No-show Status')
    plt.ylabel('Age')
    plt.show()


# Running the analysis function
plot_age_vs_no_show()
```

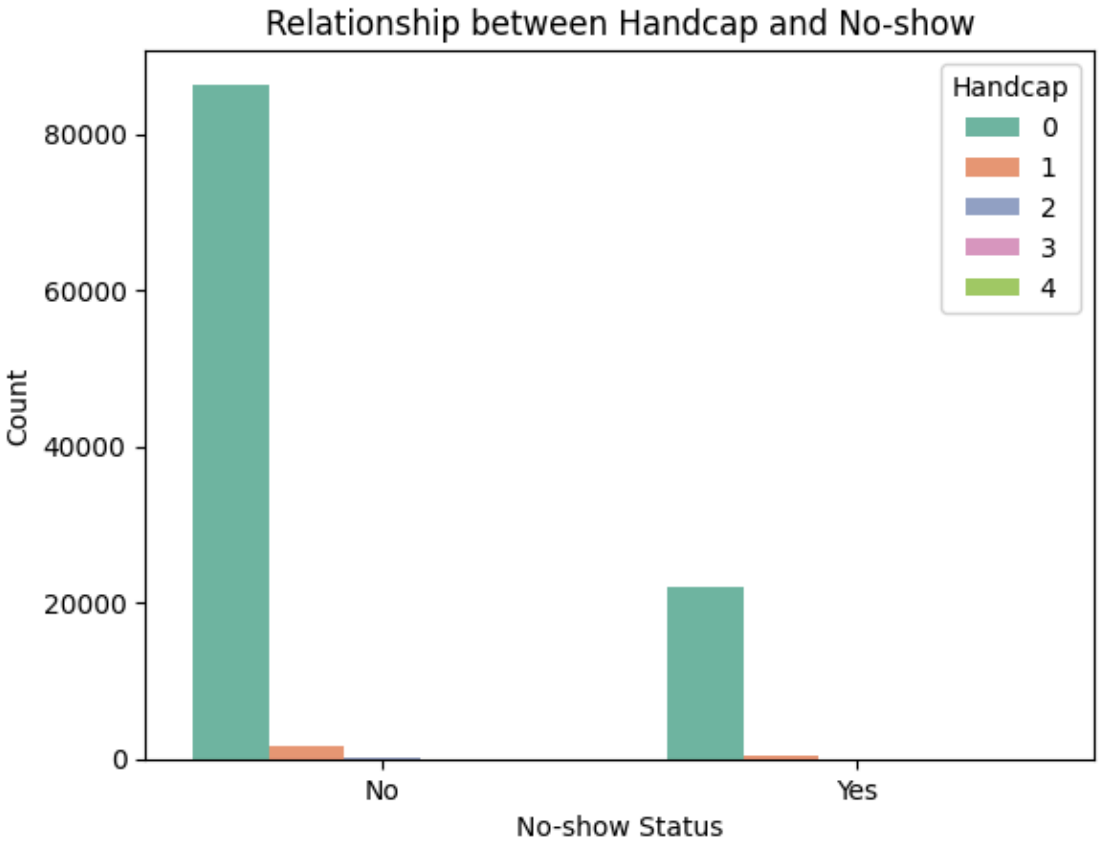


```python
# Exploratory Data Analysis Function
def plot_medical_conditions_vs_no_show():
    """Plot relationship between medical conditions and No-show status"""
    plt.figure(figsize=(12, 6))
    medical_conditions = ['Hipertension', 'Diabetes', 'Alcoholism', 'Handcap']

    for condition in medical_conditions:
        plt.figure()
        sns.countplot(x='No-show', hue=condition, data=data, palette="Set2")
        plt.title(f'Relationship between {condition} and No-show')
        plt.ylabel('Count')
        plt.xlabel('No-show Status')
        plt.legend(title=condition)
        plt.show()


# Running the analysis function
plot_medical_conditions_vs_no_show()
```

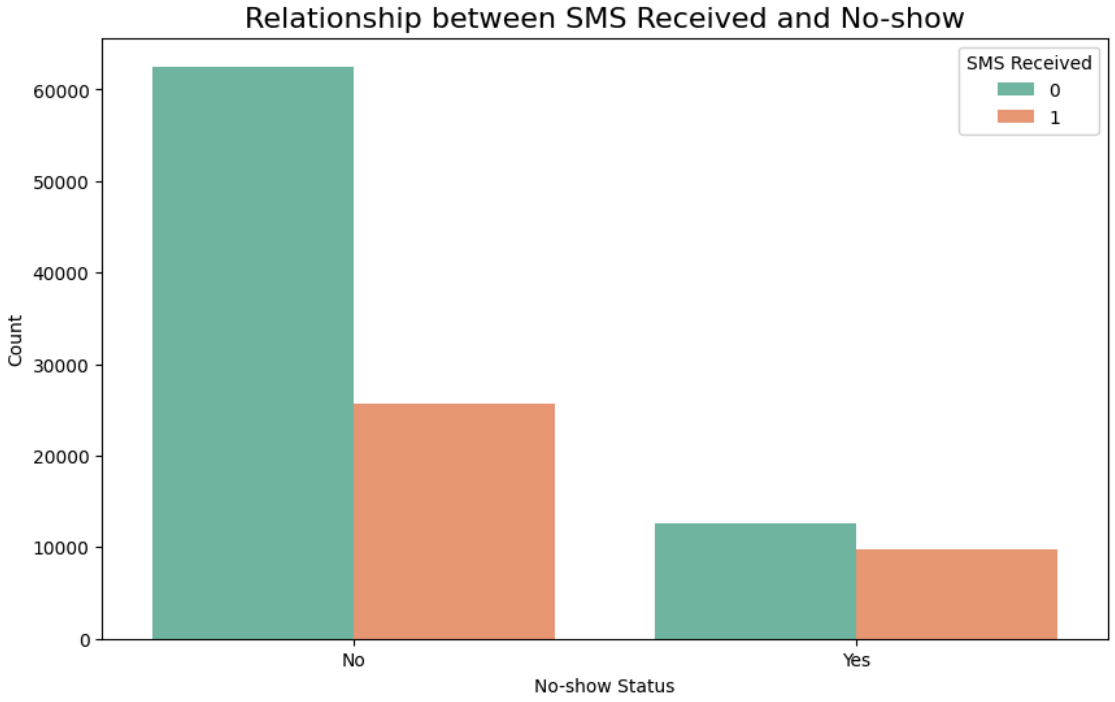



Relationship between Diabetes and No-show
Diabetes
0
1
Count
80000
70000
60000
50000
40000
30000
20000
10000
0
No
Yes
No-show Status


Relationship between Alcoholism and No-show
Alcoholism
0
1
Count
80000
70000
60000
50000
40000
30000
20000
10000
0
No
Yes
No-show Status




```python
# Exploratory Data Analysis Function
def plot_sms_vs_no_show():
    """Plot relationship between SMS reminders and No-show status"""
    plt.figure(figsize=(10, 6))
    sns.countplot(x='No-show', hue='SMS_received', data=data, palette="Set2")
    plt.title('Relationship between SMS Received and No-show', fontsize=16)
    plt.ylabel('Count')
    plt.xlabel('No-show Status')
    plt.legend(title='SMS Received', loc='upper right')
    plt.show()


# Running the analysis function
plot_sms_vs_no_show()
```



Students have to perform all the tasks illustrated above by choosing any other time series related dataset.

Implementation details:

Output:

```
[18] import pandas as pd
     import matplotlib.pyplot as plt
     import seaborn as sns
     from prophet import Prophet
```

```
# Load the dataset
file_path = 'airquality.csv'  # replace with your actual file path
air_quality_data = pd.read_csv(file_path)
air_quality_data.head()
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
0	10-03-2004	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	1692	1268	13.6	48.9	0.7578
1	10-03-2004	19:00:00	2.0	1292	112	9.4	955	103	1174	92	1559	972	13.3	47.7	0.7255
2	10-03-2004	20:00:00	2.2	1402	88	9.0	939	131	1140	114	1555	1074	11.9	54.0	0.7502
3	10-03-2004	21:00:00	2.2	1376	80	9.2	948	172	1092	122	1584	1203	11.0	60.0	0.7867
4	10-03-2004	22:00:00	1.6	1272	51	6.5	836	131	1205	116	1490	1110	11.2	59.6	0.7888

```
# Convert 'Date' and 'Time' to datetime format
air_quality_data['DateTime'] =
pd.to_datetime(air_quality_data['Date'] + ' ' +
air_quality_data['Time'], format='%d-%m-%Y %H:%M:%S')
air_quality_data.head()
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH	DateTime
0	10-03-2004	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	1692	1268	13.6	48.9	0.7578	2004-03-10 18:00:00
1	10-03-2004	19:00:00	2.0	1292	112	9.4	955	103	1174	92	1559	972	13.3	47.7	0.7255	2004-03-10 19:00:00
2	10-03-2004	20:00:00	2.2	1402	88	9.0	939	131	1140	114	1555	1074	11.9	54.0	0.7502	2004-03-10 20:00:00
3	10-03-2004	21:00:00	2.2	1376	80	9.2	948	172	1092	122	1584	1203	11.0	60.0	0.7867	2004-03-10 21:00:00
4	10-03-2004	22:00:00	1.6	1272	51	6.5	836	131	1205	116	1490	1110	11.2	59.6	0.7888	2004-03-10 22:00:00

```
# Create new features: Year, Month, Day, Hour, Weekday
air_quality_data['Year'] = air_quality_data['DateTime'].dt.year
air_quality_data['Month'] = air_quality_data['DateTime'].dt.month
air_quality_data['Day'] = air_quality_data['DateTime'].dt.day
air_quality_data['Hour'] = air_quality_data['DateTime'].dt.hour
air_quality_data['Weekday'] =
air_quality_data['DateTime'].dt.day_name()
air_quality_data.head()
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	...	PT08.S5(O3)	T	RH	AH	DateTime	Year	Month	Day	Hou
0	10-03-2004	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	...	1268	13.6	48.9	0.7578	2004-03-10 18:00:00	2004	3	10	1
1	10-03-2004	19:00:00	2.0	1292	112	9.4	955	103	1174	92	...	972	13.3	47.7	0.7255	2004-03-10 19:00:00	2004	3	10	1
2	10-03-2004	20:00:00	2.2	1402	88	9.0	939	131	1140	114	...	1074	11.9	54.0	0.7502	2004-03-10 20:00:00	2004	3	10	2
3	10-03-2004	21:00:00	2.2	1376	80	9.2	948	172	1092	122	...	1203	11.0	60.0	0.7867	2004-03-10 21:00:00	2004	3	10	2
4	10-03-2004	22:00:00	1.6	1272	51	6.5	836	131	1205	116	...	1110	11.2	59.6	0.7888	2004-03-10 22:00:00	2004	3	10	2

5 rows × 21 columns

```
# Forecasting CO(GT) concentrations using Prophet
# Step 1: Preprocess the data to aggregate daily CO concentrations
air_quality_daily = air_quality_data.groupby(air_quality_data['DateTime'].dt.date)['CO(GT)'].mean().reset_index()
air_quality_daily.columns = ['ds', 'y']

# Ensure the 'ds' column is in datetime format and without timezone
air_quality_daily['ds'] = pd.to_datetime(air_quality_daily['ds']).dt.tz_localize(None)

# Ensure 'y' (CO levels) is numeric
air_quality_daily['y'] = pd.to_numeric(air_quality_daily['y'], errors='coerce')

# Drop any rows with missing values (NaNs)
air_quality_daily = air_quality_daily.dropna()
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	...	PT08.S5(O3)	T	RH	AH	DateTime	Year	Month	Day	Ho
0	10-03-2004	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	...	1268	13.6	48.9	0.7578	2004-03-10 18:00:00	2004	3	10	
1	10-03-2004	19:00:00	2.0	1292	112	9.4	955	103	1174	92	...	972	13.3	47.7	0.7255	2004-03-10 19:00:00	2004	3	10	
2	10-03-2004	20:00:00	2.2	1402	88	9.0	939	131	1140	114	...	1074	11.9	54.0	0.7502	2004-03-10 20:00:00	2004	3	10	
3	10-03-2004	21:00:00	2.2	1376	80	9.2	948	172	1092	122	...	1203	11.0	60.0	0.7867	2004-03-10 21:00:00	2004	3	10	
4	10-03-2004	22:00:00	1.6	1272	51	6.5	836	131	1205	116	...	1110	11.2	59.6	0.7888	2004-03-10 22:00:00	2004	3	10	

5 rows × 21 columns

```
# Step 2: Initialize Prophet model for forecasting
model = Prophet()
```

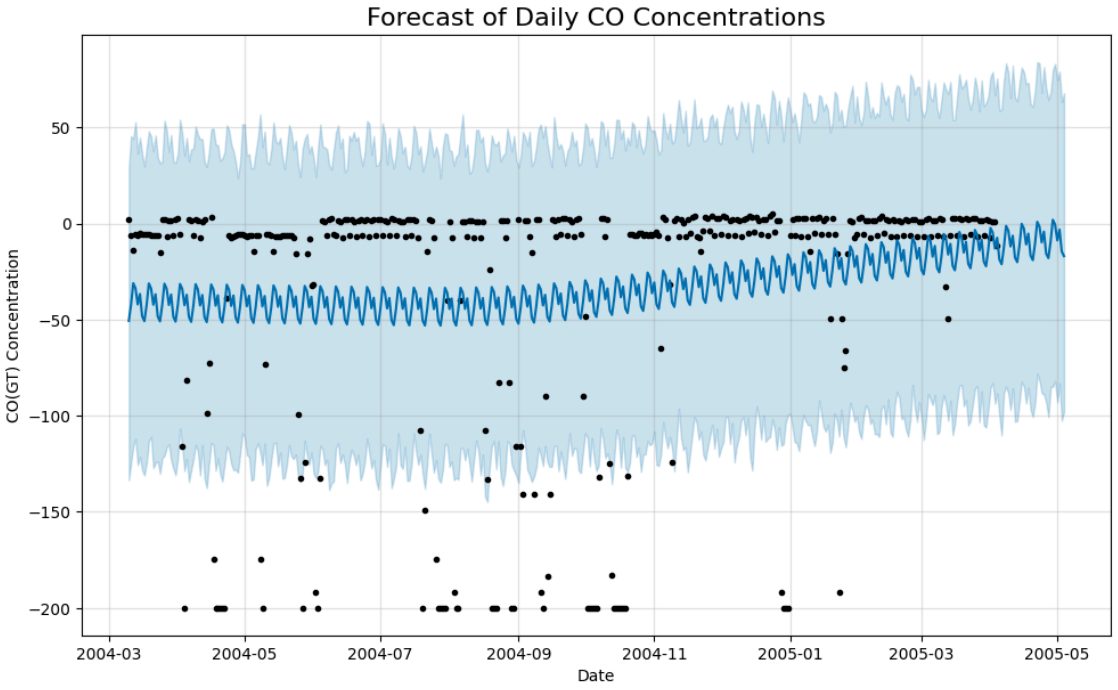
```
# Step 3: Fit the model
model.fit(air_quality_daily)
```

```
INFO:prophet:Disabling yearly seasonality. Run prophet with yearly_seasonality=True to override this.
INFO:prophet:Disabling daily seasonality. Run prophet with daily_seasonality=True to override this.
DEBUG:cmdstanpy:input tempfile: /tmp/tmpdzgvvyur/796gbloz.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpdzgvvyur/saetqjc7.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.10/dist-packages/prophet/stan_model/prophet_model.bin', 'random', 'seed=14913', 'data', 'file=/tmp/tmpdzgvvyur/796gb
07:10:37 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
07:10:37 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
<prophet.forecaster.Prophet at 0x7afb260b89d0>
```

```
# Step 4: Create future dates for prediction (e.g., next 30 days)
future_dates = model.make_future_dataframe(periods=30)
```

```
# Step 5: Predict future CO concentrations
forecast = model.predict(future_dates)
```

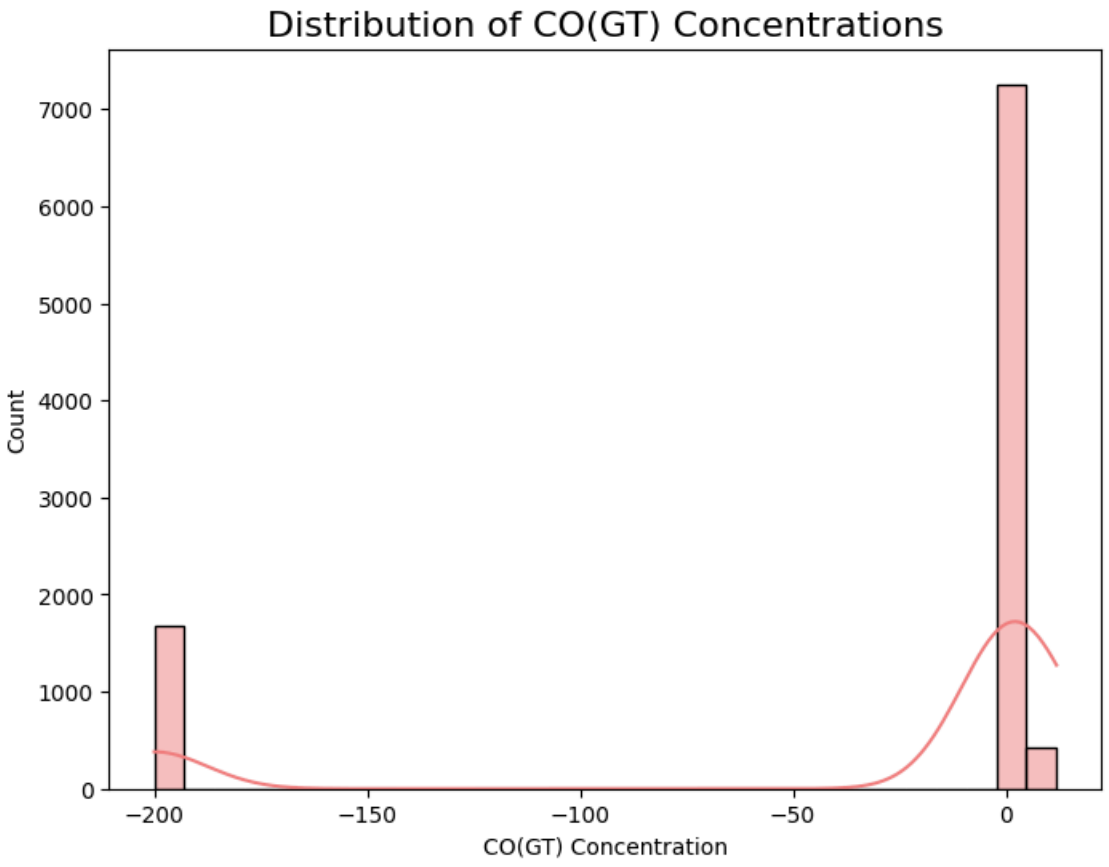
```
# Step 6: Plot the forecast
fig = model.plot(forecast)
plt.title('Forecast of Daily CO Concentrations', fontsize=16)
plt.xlabel('Date')
plt.ylabel('CO(GT) Concentration')
plt.show()
```

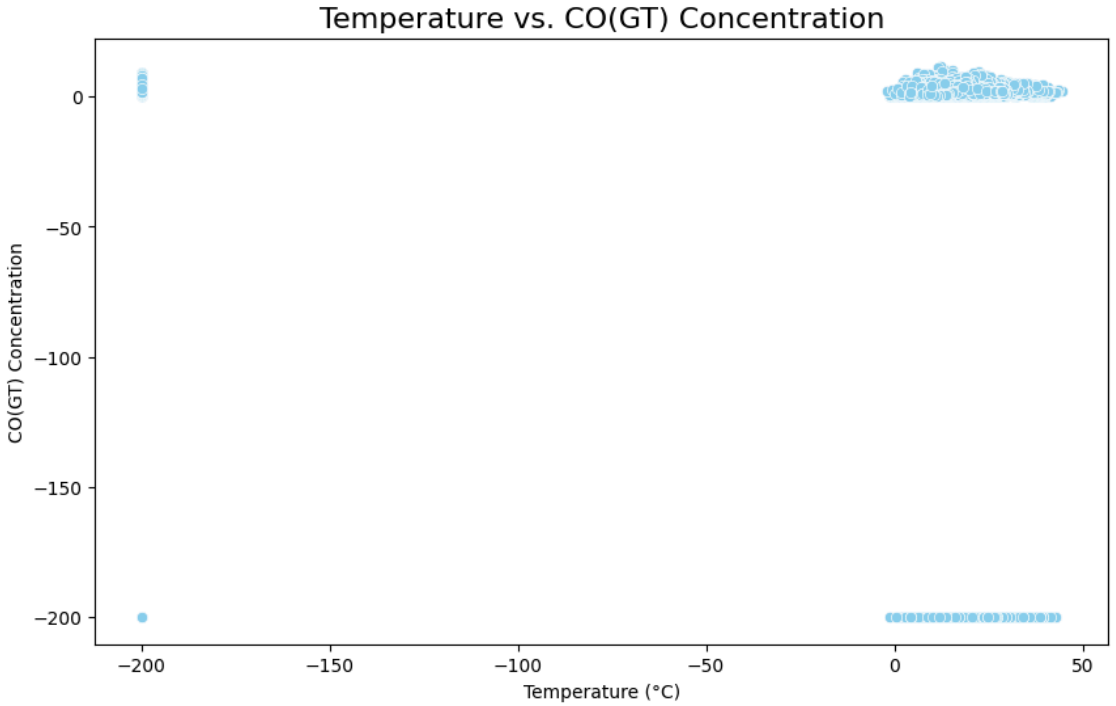


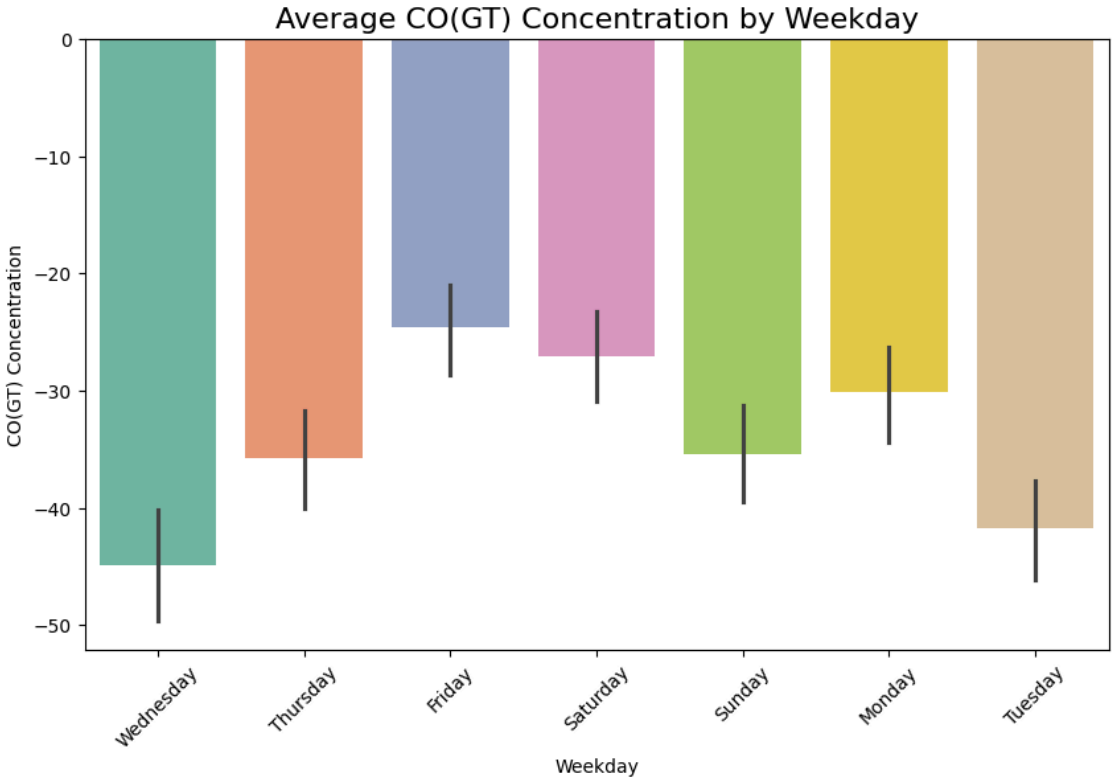

```
# Exploratory Data Analysis (EDA)

def plot_co_distribution():
    """Plot distribution of CO concentrations"""
    plt.figure(figsize=(8, 6))
       sns.histplot(air_quality_data['CO(GT)'], bins=30, kde=True,
color='lightcoral')
    plt.title('Distribution of CO(GT) Concentrations', fontsize=16)
    plt.xlabel('CO(GT) Concentration')
    plt.ylabel('Count')
    plt.show()

def plot_temperature_vs_co():
         """Plot relationship between Temperature and CO(GT)
Concentration"""
    plt.figure(figsize=(10, 6))
        sns.scatterplot(x='T', y='CO(GT)', data=air_quality_data,
color='skyblue')
    plt.title('Temperature vs. CO(GT) Concentration', fontsize=16)
    plt.xlabel('Temperature (°C)')
    plt.ylabel('CO(GT) Concentration')
```

```
    plt.show()

def plot_weekday_vs_co():
    """Plot average CO(GT) Concentration by Weekday"""
    plt.figure(figsize=(10, 6))
       sns.barplot(x='Weekday', y='CO(GT)', data=air_quality_data,
estimator=lambda x: x.mean(), palette="Set2")
          plt.title('Average CO(GT) Concentration by Weekday',
fontsize=16)
    plt.ylabel('CO(GT) Concentration')
    plt.xticks(rotation=45)
    plt.show()
```

```
# Running the EDA functions
plot_co_distribution()
plot_temperature_vs_co()
plot_weekday_vs_co()
```







Date: 24 October 2024 **Signature of faculty in-charge**

Post Lab Descriptive Questions:

1. ***Explain the components of time series?***

Time series data typically consists of several key components that help in understanding its underlying patterns and trends. Here are the main components:

- Trend: This represents the long-term movement or direction in the data over time. Trends can be upward, downward, or stable.
- Seasonality: This refers to regular, predictable patterns that occur at specific intervals, such as daily, monthly, or yearly. Seasonality often correlates with calendar events or seasons.
- Cyclic Patterns: Unlike seasonality, cyclic patterns are long-term fluctuations that occur over irregular intervals, often linked to economic or business cycles.
- Irregular or Random Component: This represents random noise or unpredictable events that do not follow a discernible pattern. These fluctuations can be due to unexpected occurrences, such as natural disasters or sudden market changes.
- Level: The average value around which the time series fluctuates. It can be thought of as the baseline or central tendency of the data.

2. ***How do you handle seasonality in time series data? What methods or transformations can you apply?***

Following are several methods and transformations you can apply:

1. Seasonal Decomposition:
    a. Additive Decomposition: Decomposes the time series into trend, seasonal, and irregular components, assuming the seasonal component is constant over time.
    b. Multiplicative Decomposition: Similar to additive, but assumes that the seasonal effect varies with the level of the series.

2. Differencing:
    a. Seasonal Differencing: Subtracting the value from the same season in the previous cycle (e.g., the same month last year) to remove seasonality.
3. Seasonal Adjustment:
    a. Use methods like X-12-ARIMA or STL (Seasonal-Trend decomposition using LOESS) to adjust the data for seasonality.
4. Fourier Transforms:
    a. Apply Fourier transforms to capture seasonal patterns by adding sine and cosine terms corresponding to the seasonal frequencies.
5. Dummy Variables:
    a. Create dummy variables for seasonal periods (e.g., months or quarters) and include them in regression models to account for seasonal effects.
6. Moving Averages:
    a. Apply moving averages to smooth out seasonal fluctuations, which can help in identifying trends.
7. Exponential Smoothing:
    a. Methods like Holt-Winters seasonal smoothing explicitly account for seasonality, trend, and level in the forecasts.
8. Machine Learning Models:
    a. Use models like seasonal ARIMA, SARIMA, or advanced machine learning techniques that can naturally capture seasonal patterns.

3. What are some common metrics for evaluating forecasting models (e.g., MAE, RMSE, MAPE)?

1. Mean Absolute Error (MAE):
    - Measures the average absolute difference between forecasted and actual values.

- Formula: MAE=1n∑t=1n|yt−y^t|\text{MAE} = \frac{1}{n} \sum_{t=1}^{n} |y_t - \hat{y}_t|MAE=n1t=1∑n|yt−y^t|
- Advantages: Easy to interpret; treats all errors equally.

2. Root Mean Squared Error (RMSE):
   - Calculates the square root of the average squared differences between forecasted and actual values.
   - Formula: RMSE=1n∑t=1n(yt−y^t)2\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^{n} (y_t - \hat{y}_t)^2}RMSE=n1t=1∑n(yt−y^t)2
   - Advantages: Penalizes larger errors more heavily; sensitive to outliers.
3. Mean Absolute Percentage Error (MAPE):
   - Measures the average absolute percentage error between forecasted and actual values.
   - Formula: MAPE=100n∑t=1n|yt−y^tyt|\text{MAPE} = \frac{100}{n} \sum_{t=1}^{n} \left|\frac{y_t - \hat{y}_t}{y_t}\right|MAPE=n100t=1∑nytyt−y^t
   - Advantages: Scale-independent; expressed as a percentage, making it easy to interpret.
4. Mean Squared Error (MSE):
   - Similar to RMSE but without taking the square root, measuring the average of the squared differences.
   - Formula: MSE=1n∑t=1n(yt−y^t)2\text{MSE} = \frac{1}{n} \sum_{t=1}^{n} (y_t - \hat{y}_t)^2MSE=n1t=1∑n(yt−y^t)2
   - Advantages: Useful for highlighting the magnitude of errors.
5. Theil's U Statistic:
   - Compares the accuracy of the forecasting model to a naïve benchmark.
   - Values less than 1 indicate that the model is more accurate than the naïve approach.
6. R-squared (Coefficient of Determination):
   - Measures the proportion of variance in the dependent variable explained by the model.

- Useful in regression-based forecasts.

7. Tracking Signal:
   - Helps detect bias in forecasts by calculating the cumulative forecast error relative to the average error.